



Explore | Expand | Enrich

<Codemithra />TM

PRIORITY QUEUE USING DLL



PRIORITY QUEUE USING DLL

EXPLANATION

A Priority Queue is a data structure that maintains a set of elements, each associated with a priority.

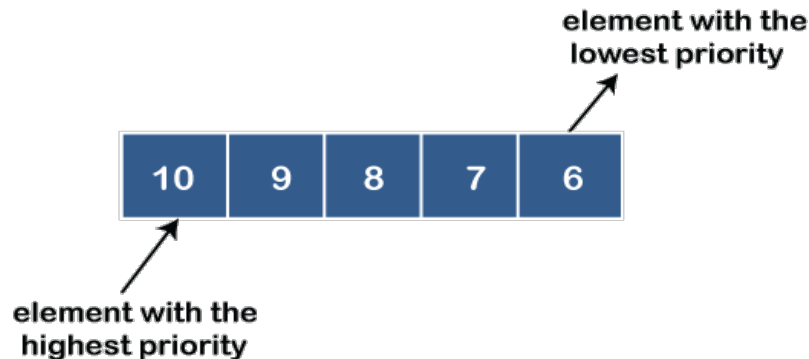
The basic idea is that elements with higher priority are served before elements with lower priority.

Priority Queues are often implemented using various data structures, and one common approach is to use a doubly linked list (DLL) to implement a Priority Queue.

PRIORITY QUEUE USING DLL

EXPLANATION

A Doubly Linked List is a data structure in which each node contains a data element and two pointers, one pointing to the next node and another pointing to the previous node.



PRIORITY QUEUE USING DLL

EXPLANATION

A priority queue is a data structure that organizes elements based on their assigned priorities.

While traditional queues follow the first-in-first-out (FIFO) rule, priority queues prioritize elements based on certain criteria.

The priority can be assigned according to the element's value, with the lowest or highest values having the highest priority.

Alternatively, priorities can be set based on specific requirements or criteria

PRIORITY QUEUE USING DLL

EXPLANATION

Priority queues can be implemented using various data structures, such as arrays, linked lists, heaps, or binary search trees.

Operations:

1. `push()`: Adds an element to the priority queue with its associated priority.
2. `pop()`: Removes and returns the element with the highest priority from the priority queue.
3. `peek()`: Returns the element with the highest priority without removing it.

PRIORITY QUEUE USING DLL

APPROACH

1. Create a doubly linked list node:

Define a class named ``Node`` with fields: ``info``, ``priority``, ``prev``, and ``next``.

2. Insert the element and priority in the Node:

Instantiate a new ``Node`` and set its ``info`` and ``priority`` fields with the provided element and priority.

PRIORITY QUEUE USING DLL

APPROACH

3. Arrange the Nodes in increasing order of priority:

Check conditions for insertion:

If the list is empty, set `front` and `rear` to the new Node.

If the priority is less than or equal to the priority of the front Node, insert the new Node at the front.

If the priority is greater than the priority of the rear Node, insert the new Node after the rear.

Otherwise, find the position based on priority and insert the new Node accordingly.

This approach ensures that the linked list is maintained in ascending order of priority as elements are inserted.

PRIORITY QUEUE USING DLL

APPROACH

3. Arrange the Nodes in increasing order of priority:

Check conditions for insertion:

If the list is empty, set `front` and `rear` to the new Node.

If the priority is less than or equal to the priority of the front Node, insert the new Node at the front.

If the priority is greater than the priority of the rear Node, insert the new Node after the rear.

Otherwise, find the position based on priority and insert the new

Node accordingly

PRIORITY QUEUE USING DLL

```
import java.util.*;
class Main {
    static Node front, rear;
    static class Node {
        int info;
        int priority;
        Node prev, next;
    }
    static void push(Node fr, Node rr, int n,
int p) {
        Node news = new Node();
        news.info = n;
        news.priority = p;
        if (fr == null) {
            fr = news;
            rr = news;
            news.next = null;
        } else {
            if (p <= (fr).priority) {
                news.next = fr;
```

```
(fr).prev = news.next;
            fr = news;
        }
        else if (p > (rr).priority) {
            news.next = null;
            (rr).next = news;
            news.prev = (rr).next;
            rr = news;
        }
        else {
            Node start = (fr).next;
            while (start.priority > p)
                start = start.next;
            (start.prev).next = news;
            news.next = start.prev;
            news.prev = (start.prev).next;
            start.prev = news.next;
        }
    }
}
```



PRIORITY QUEUE USING DLL

```
front = fr;
rear = rr;
}
static int peek(Node fr) {
    return fr.info;
}
static boolean isEmpty(Node fr) {
    return (fr == null);
}
static int pop(Node fr, Node rr) {
    Node temp = fr;
    int res = temp.info;
    (fr) = (fr).next;
    if (fr == null)
        rr = null;
    front = fr;
    rear = rr;
    return res;
}
```

```
public static void main(String args[]) {
    push(front, rear, 2, 3);
    push(front, rear, 3, 4);
    push(front, rear, 4, 5);
    push(front, rear, 5, 6);
    push(front, rear, 6, 7);
    push(front, rear, 1, 2);
    System.out.println(pop(front, rear));
    System.out.println(peek(front));
}
}
```



PRIORITY QUEUE USING DLL

TIME AND SPACE COMPLEXITY

Time Complexity: $O(n)$ - where n is the number of elements in the priority queue (assuming the linked list is singly linked and elements are inserted in sorted order based on priority).

Space Complexity: $O(1)$ - constant space as the linked list is modified in-place without using additional data structures proportional to the input size.

INTERVIEW QUESTIONS

1. How does the priority queue using DLL handle elements with the same priority?

Answer: Elements with the same priority are inserted in the order they arrive, maintaining their relative order.

INTERVIEW QUESTIONS

2. What is the time complexity of the enqueue operation in the DLL-based priority queue?

Answer: The time complexity of the enqueue operation is $O(n)$, where n is the number of elements in the priority queue.

INTERVIEW QUESTIONS

3. How does the dequeue operation work in the DLL-based priority queue?

Answer: The dequeue operation removes and returns the element with the highest priority (lowest numerical priority value).

INTERVIEW QUESTIONS

4. Can the priority queue handle elements of any data type?

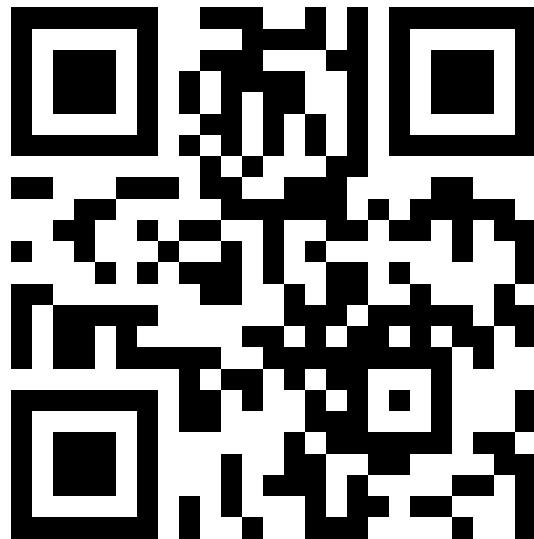
Answer: The implementation provided in the answer is specific to integers. To handle elements of any data type, you can use Java generics.



INTERVIEW QUESTIONS

5. What is the advantage of using a Doubly Linked List for implementing a priority queue?

Answer: The advantage of using a Doubly Linked List is that it allows efficient insertion and removal of elements based on priority, especially when compared to a singly linked list. This is because DLLs support traversal in both directions, making it easier to find the correct position for insertion or removal.



<https://learn.codemithra.com>



THANK YOU



+91 78150 95095



codemithra@ethnus.com



www.codemithra.com